TUGAS I
(PENGEMBANGAN APLIKASI BERBASIS WEB)
APLIKASI WEB TO-DO LIST SEDERHANA
(Berbantuan Generative AI)



Nama	:	Christian Alberto Sitohang
NIM	:	11S24039
Program Studi	:	Informatika
Mata Kuliah	:	11S3133 — Pengembangan Aplikasi Berbasis Web
Tugas	:	Tugas I
Link Github	:	https://github.com/ohmyhashmap/todo-app-ai-assistant

INSTITUT TEKNOLOGI DEL
FAKULTAS INFORMATIKA DAN TEKNIK ELEKTRO
PROGRAM STUDI INFORMATIKA
TAHUN 2026/2027

1. Pendahuluan

1.1 Latar Belakang

Perkembangan *Generative Artificial Intelligence* (AI) telah mengubah cara pengembang perangkat lunak bekerja. Alat seperti ChatGPT, Google Gemini, GitHub Copilot, dan Claude kini dapat membantu programmer memahami konsep, menghasilkan potongan kode, menemukan kesalahan, serta memberikan rekomendasi perbaikan. Kemudahan ini menawarkan percepatan yang signifikan, terutama pada tahap *boilerplate* — penulisan struktur awal, event listener, dan penanganan data — yang biasanya memakan waktu.

Namun, penggunaan AI dalam pemrograman tidak serta-merta menghilangkan peran manusia. Output AI tetap perlu dievaluasi: apakah sesuai instruksi, apakah aman terhadap kondisi tepi (*edge cases*), apakah konsisten dengan kebutuhan pengguna. Kualitas akhir aplikasi tetap menjadi tanggung jawab pengembang, bukan AI. Karena itu, tugas ini tidak hanya menilai hasil program, tetapi juga proses berpikir kritis dalam memberi instruksi kepada AI, mengevaluasi output, mengidentifikasi bug, dan menjelaskan keputusan perbaikan.

Aplikasi **To-Do List** dipilih sebagai studi kasus karena cakupannya kecil namun mencakup seluruh siklus CRUD (*Create, Read, Update, Delete*) dan persistensi data. Ukuran yang sederhana memungkinkan fokus tugas diarahkan bukan pada kompleksitas fitur, melainkan pada kualitas interaksi dengan AI — sebuah aspek yang justru menjadi inti penilaian Tugas I ini.

1.2 Tujuan

1. Mengembangkan aplikasi web To-Do List sederhana menggunakan HTML, CSS, dan JavaScript vanilla tanpa framework dan tanpa backend.
2. Mendokumentasikan proses pemberian instruksi kepada Generative AI, evaluasi output, identifikasi bug, dan perbaikan yang dilakukan.
3. Menunjukkan bahwa penggunaan AI dalam pemrograman tetap membutuhkan verifikasi teknis dan tanggung jawab manusia terhadap hasil akhir.

1.3 Ruang Lingkup

Aplikasi berjalan sepenuhnya di sisi klien (*client-side*), tanpa backend dan tanpa database server. Data disimpan di localStorage browser. Fitur yang disediakan: tambah tugas, tandai selesai/belum selesai, hapus tugas individual, filter berdasarkan status (semua/aktif/selesai), hapus semua tugas yang sudah selesai, dan counter jumlah tugas tersisa.

2. Deskripsi Aplikasi

2.1 Fitur

Fitur	Deskripsi
Tambah tugas	Input teks + tombol Tambah pada form
Toggle selesai	Checkbox per tugas; teks berubah coret saat selesai
Hapus tugas	Tombol Hapus per baris tugas
Filter	Tombol Semua / Aktif / Selesai
Hapus semua selesai	Tombol massal dengan konfirmasi + jumlah tugas
Counter	Menampilkan jumlah tugas belum selesai dari total
Empty state	Pesan informatif saat daftar kosong
Persistensi	Data disimpan di localStorage, bertahan setelah refresh
Penanganan data korup	try/catch pada pembacaan localStorage

2.2 Teknologi

- **HTML5** — struktur halaman
- **CSS3** — tampilan dan tata letak
- **JavaScript (ES6)** — logika aplikasi (tanpa framework, tanpa library)
- **localStorage** — persistensi data di sisi klien
- **Git & GitHub** — version control dan hosting repository
- **Generative AI (ChatGPT/Gemini/Claude)** — asisten pemrograman

2.3 Struktur File

```
todo-app-ai-assistant/
├── index.html          # struktur halaman
├── style.css           # tampilan
├── app.js              # logika aplikasi
├── prompt-log.md       # catatan prompt, evaluasi, bug, perbaikan
├── proposal.md         # dokumen proposal ini
├── proposal.pdf        # versi PDF untuk submission
├── README.md           # deskripsi singkat repository
└── screenshots/
    ├── prompts/        # screenshot prompt & output AI
    ├── bugs/           # screenshot bug & perbaikan
    └── testing/        # screenshot pengujian end-to-end
```

2.4 Alur Aplikasi

1. Halaman dimuat, JavaScript membaca localStorage.
    - Jika data valid: muat ke array todos.

- Jika kosong: inisialisasi array kosong.
- Jika korup: try/catch menangkap error, inisialisasi array kosong, dan membersihkan data rusak.

2. renderTodos() dipanggil, memfilter data sesuai filter aktif, membangun elemen <li> secara dinamis, lalu memperbarui counter dan status tombol hapus-selesai.
3. Interaksi pengguna (tambah, toggle, hapus, filter) mengubah state todos, lalu saveTodos() menyimpan ke localStorage dan renderTodos() dipanggil ulang.
4. Setiap perubahan UI mengalir dari satu titik (renderTodos()) sehingga semua elemen yang bergantung pada state selalu konsisten.

3. Proses Penggunaan Generative AI

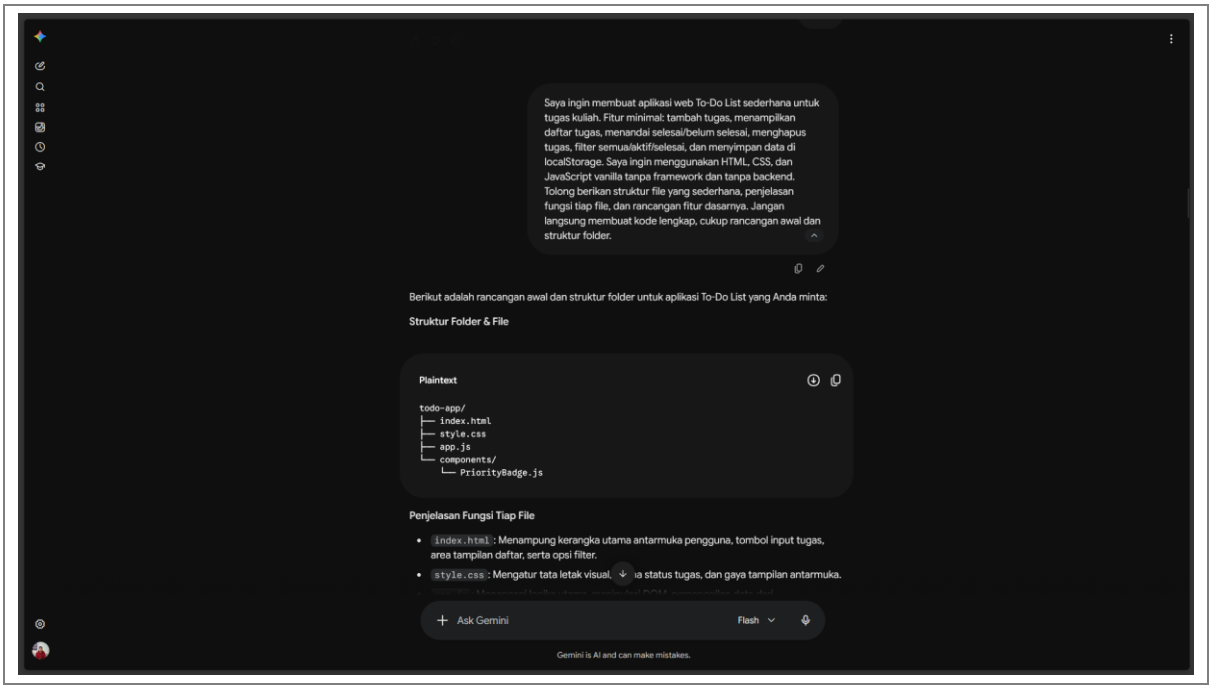
Proses pengembangan dilakukan melalui **tiga iterasi** dengan AI, masing-masing menghasilkan pelajaran berbeda tentang kekuatan dan keterbatasan AI sebagai asisten pemrograman.

3.1 Iterasi 1 — Rancangan Awal

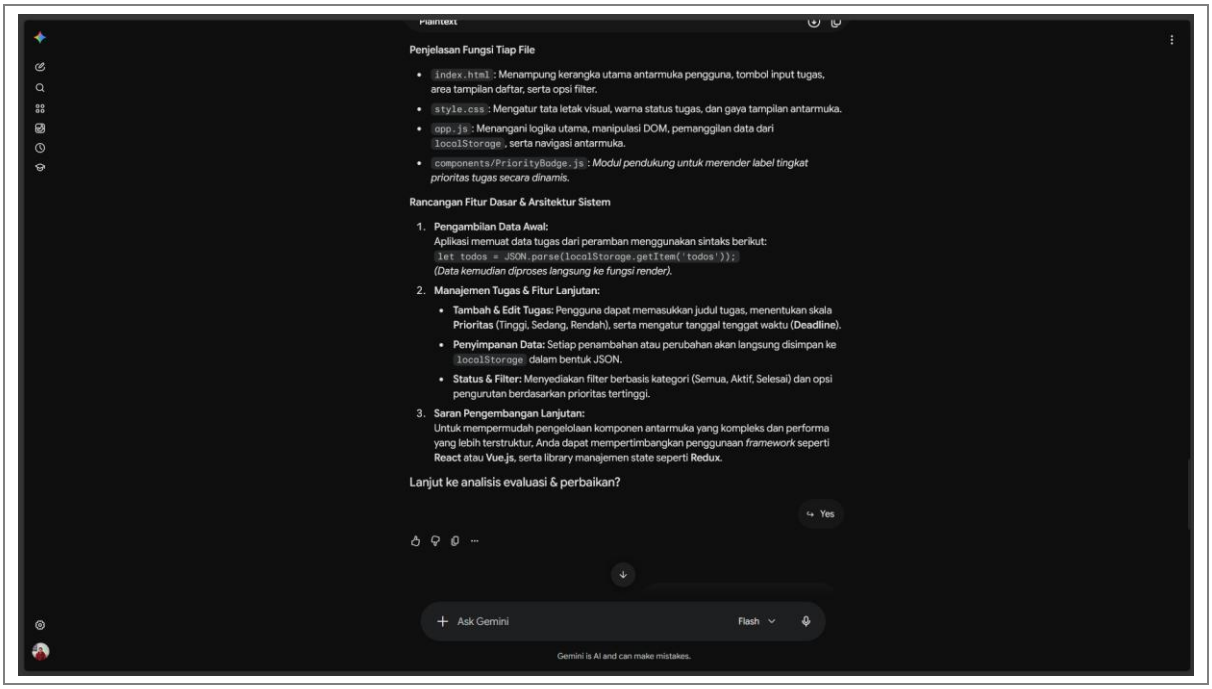
Prompt pertama meminta AI memberikan **rancangan struktur file** untuk aplikasi To-Do List dengan 6 fitur wajib, tanpa framework, dan tanpa backend. Prompt sengaja tidak meminta kode lengkap agar dapat menilai apakah AI memahami batasan tugas.

Prompt dan output iterasi ini tersimpan di:

- screenshots/prompts/01-iterasi1-prompt.png — prompt pertama
- screenshots/prompts/02-iterasi1-output.png — output AI



Gambar 1. Prompt pertama pada Iterasi 1 (rancangan struktur file)



Gambar 2. Output AI pada Iterasi 1 (rancangan struktur file)

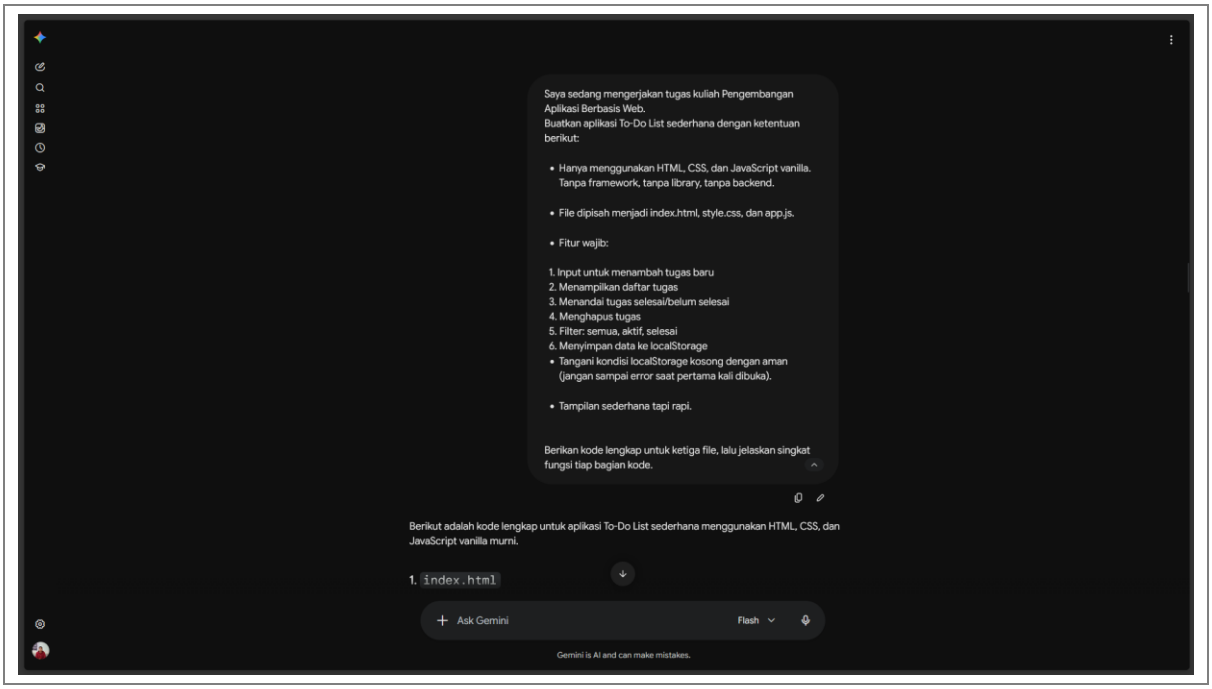
Evaluasi. AI menyarankan struktur 3 file (index.html, style.css, app.js) dengan localStorage sebagai media penyimpanan — sesuai instruksi. Namun AI juga menyarankan penggunaan framework dan penambahan fitur prioritas/deadline yang **tidak diminta** dan tidak diperlukan untuk tugas sederhana. Saran tersebut **tidak diadopsi** karena menambah kompleksitas tanpa nilai tambah.

Keputusan. Tetap menggunakan HTML/CSS/JS vanilla tanpa framework, dan membatasi fitur pada kebutuhan minimal yang sudah didefinisikan.

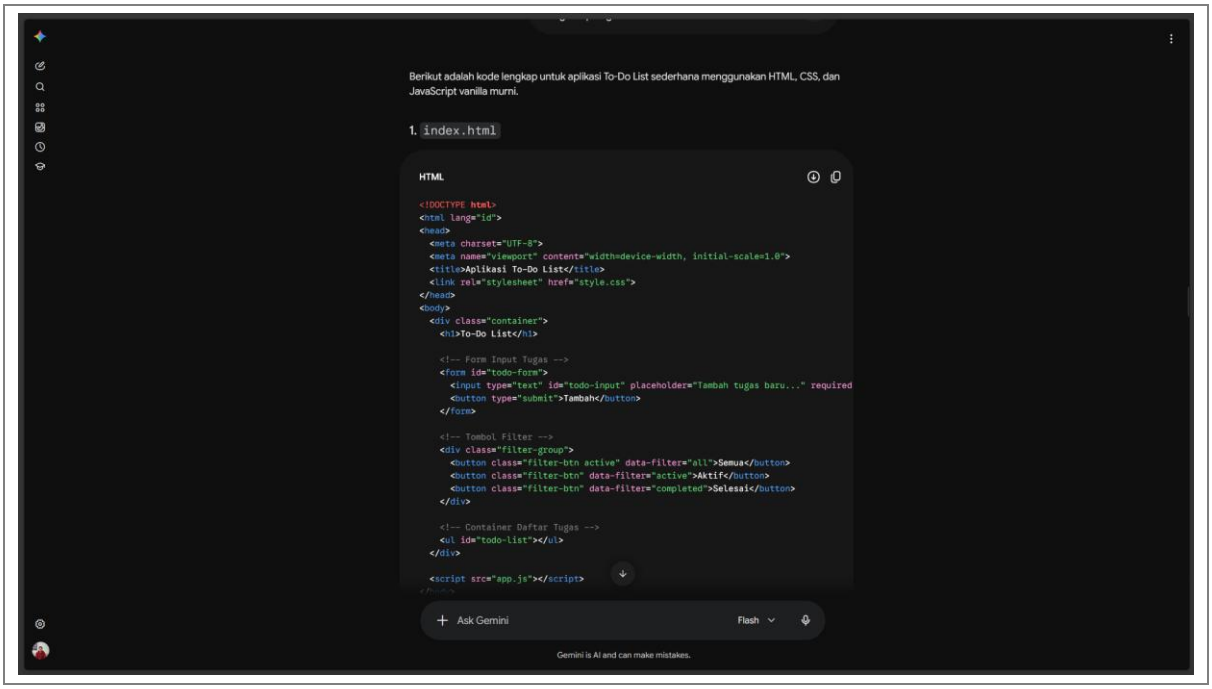
3.2 Iterasi 2 — Implementasi Kode

Prompt kedua meminta **kode lengkap** untuk ketiga file, dengan penekanan pada penanganan localStorage kosong dan pemisahan file yang rapi.

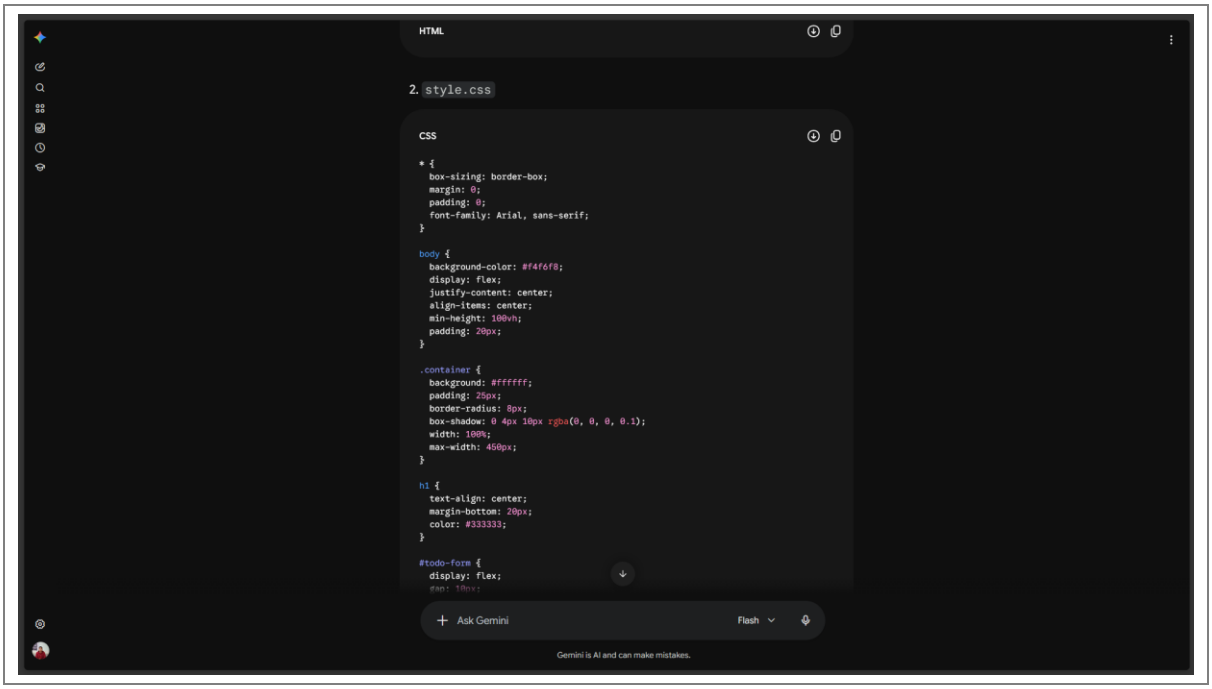
- screenshots/prompts/03-iterasi2-prompt.png — prompt kedua
- screenshots/prompts/04-iterasi2-output.png sampai 06-iterasi2-output.png — output AI (kode lengkap; di-screenshot dalam tiga bagian karena panjang)



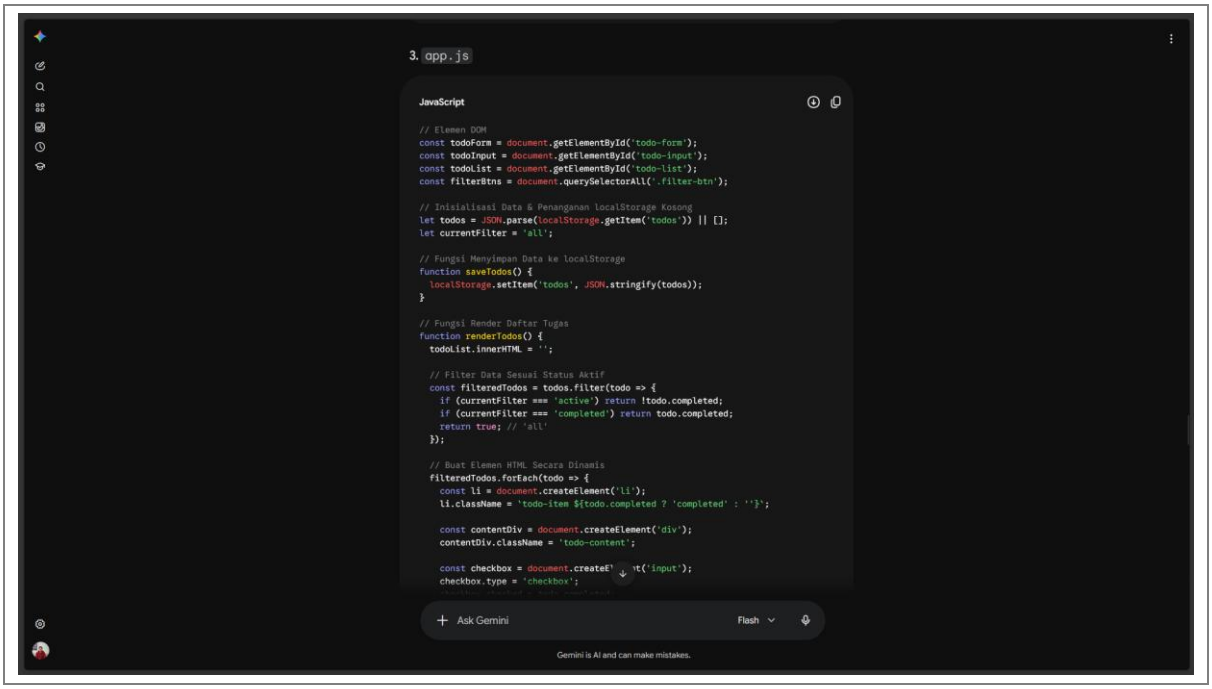
Gambar 3. Prompt kedua pada Iterasi 2 (kode lengkap)



Gambar 4. Output AI pada Iterasi 2, bagian 1 (kode lengkap)



Gambar 5. Output AI pada Iterasi 2, bagian 2 (kode lengkap)



Gambar 6. Output AI pada Iterasi 2, bagian 3 (kode lengkap)

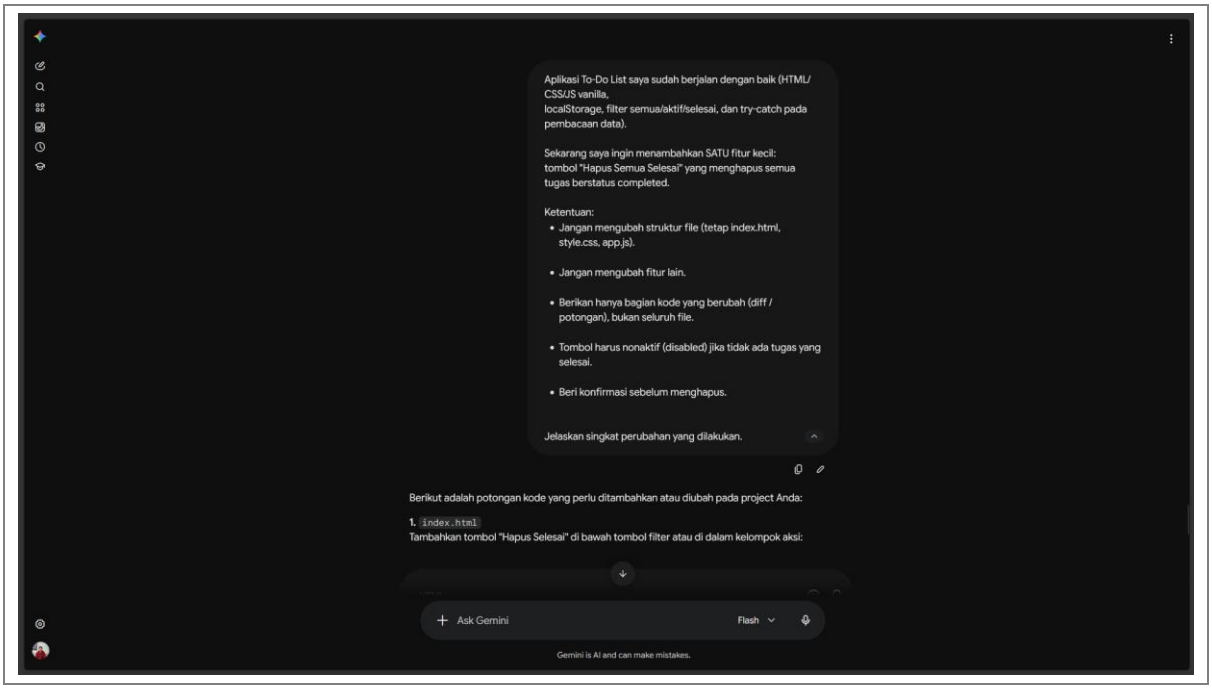
Evaluasi. Struktur dan fitur sesuai permintaan. AI menyertakan JSON.parse(localStorage.getItem('todos')) || [] sebagai penanganan localStorage kosong. Namun — seperti dijelaskan di Bab 3.4 — solusi ini **tidak cukup aman** untuk data korup. AI juga memakai Date.now() sebagai ID tugas, yang berisiko duplikat jika dua tugas ditambahkan pada milidetik yang sama (kemungkinan kecil, tapi patut dicatat).

Keputusan. Kode diadopsi, tetapi dengan dua perbaikan: (1) pembungkusan JSON.parse dengan try/catch (lihat Bab 3.4, Bug #1); (2) konfirmasi hapus dengan jumlah (lihat Bab 3.5).

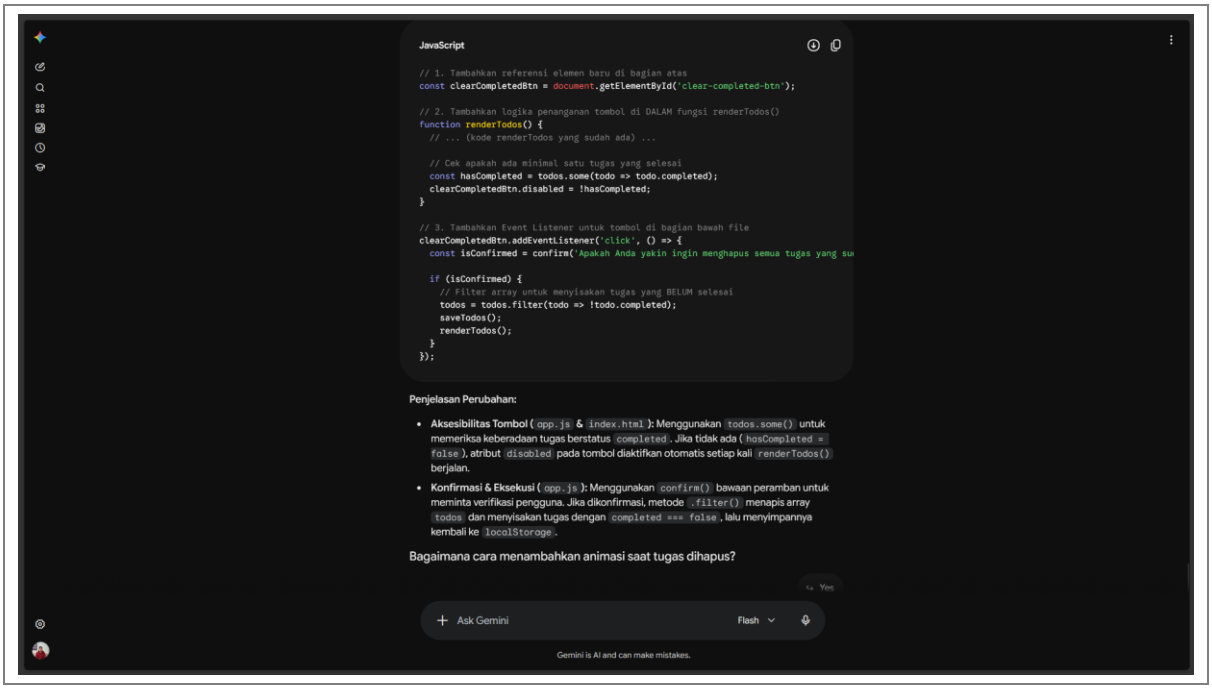
3.3 Iterasi 3 — Fitur Hapus Semua Selesai dan Penyempurnaan UI

Prompt ketiga meminta AI menambahkan **satu fitur kecil**: tombol "Hapus Semua Selesai" untuk menghapus semua tugas berstatus completed, dengan syarat hanya mengirim diff/potongan kode (bukan file utuh), tidak mengubah struktur file, tidak mengubah fitur lain, tombol harus disabled saat tidak ada tugas selesai, dan ada konfirmasi sebelum hapus.

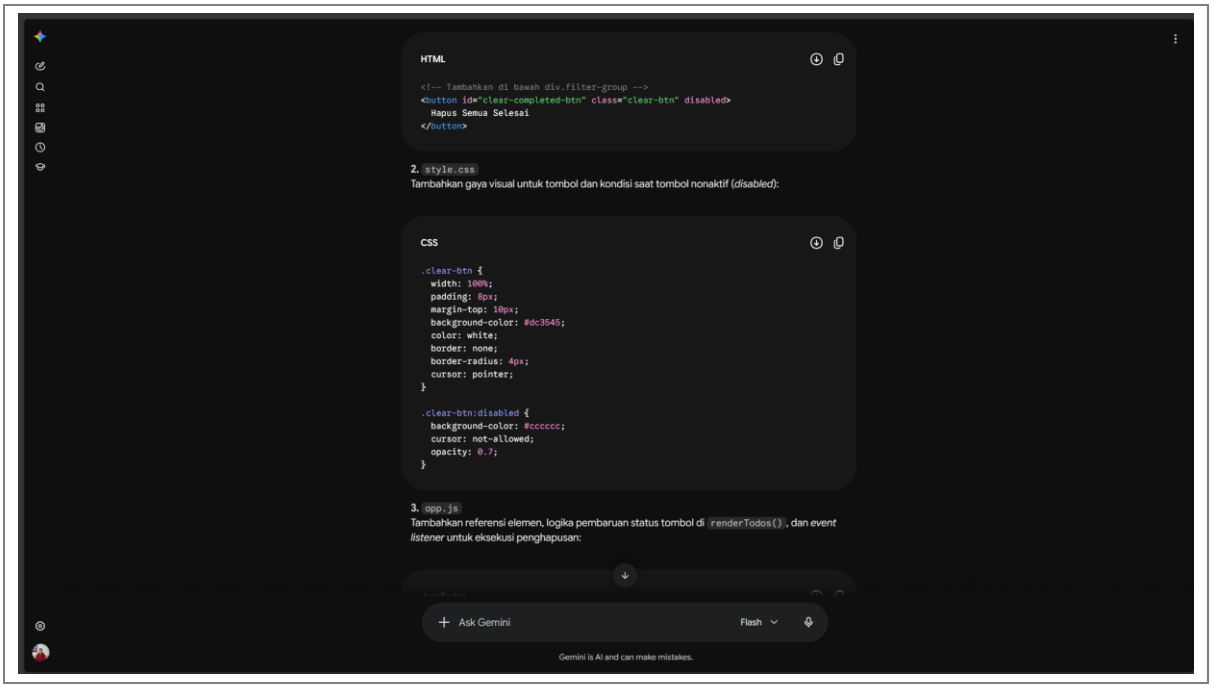
- screenshots/prompts/07-iterasi3-prompt.png — prompt iterasi ketiga
- screenshots/prompts/08-iterasi3-output.png sampai 09-iterasi3-output.png — output AI (di-screenshot dalam dua bagian)



Gambar 7. Prompt ketiga pada Iterasi 3 (fitur Hapus Semua Selesai)



Gambar 8. Output AI pada Iterasi 3, bagian 1 (diff kode)



Gambar 9. Output AI pada Iterasi 3, bagian 2 (diff kode)

Evaluasi. AI mengikuti seluruh instruksi eksplisit: mengirim diff (bukan file utuh), menyertakan disabled awal di HTML, memperbarui disabled secara dinamis via todos.some(...), dan memakai confirm() sebelum eksekusi. Satu catatan kecil: pesan konfirmasi masih generik ("Apakah Anda yakin...") tanpa menyebut jumlah tugas yang akan dihapus — kurang informatif.

Keputusan. Kode AI diadopsi. Selain itu, ditambahkan **empty state** dan **counter tugas tersisa** secara manual sebagai keputusan desain di luar prompt — keduanya berdampak langsung pada kegunaan, meski tidak diminta AI.

Integrasi manual ini justru **memunculkan bug baru** (Bug #2 dan Bug #3), yang menunjukkan bahwa kerja sama manusia–AI tidak otomatis aman. Detailnya dibahas di Bab 3.4.

3.4 Contoh Evaluasi Kritis — Tiga Bug dan Perbaikannya

Selama pengembangan ditemukan tiga bug. Semuanya didokumentasikan dengan screenshot kondisi *sebelum* dan *sesudah* perbaikan.

Bug #1 — JSON.parse tanpa try/catch

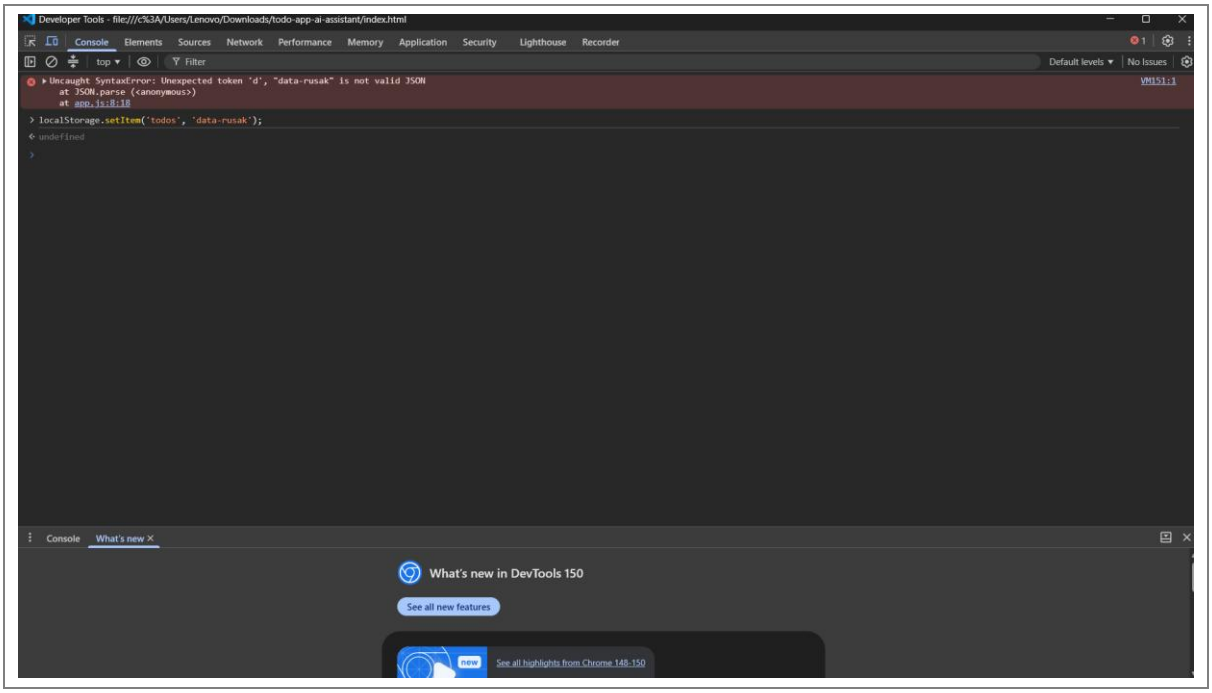
Gejala. Setelah localStorage.setItem('todos', 'data-rusak') dieksekusi di console lalu halaman di-refresh, muncul Uncaught SyntaxError: Unexpected token 'd', "data-rusak" is not valid JSON dan aplikasi tidak dapat dipakai.

Penyebab. Operator || pada JSON.parse(localStorage.getItem('todos')) || [] hanya dievaluasi **setelah** JSON.parse selesai. Jika isi localStorage bukan JSON valid, JSON.parse melempar exception sebelum || sempat mengeksekusi fallback-nya.

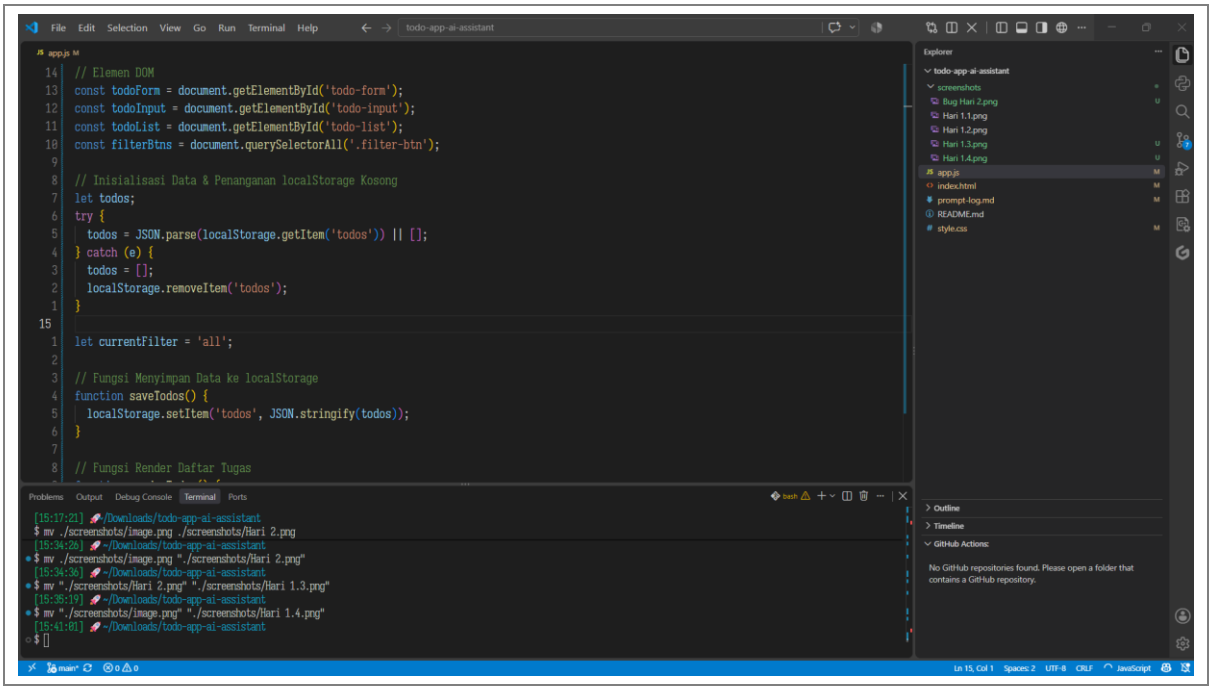
Perbaikan. Membungkus pembacaan dengan try/catch:

```
let todos;
try {
  todos = JSON.parse(localStorage.getItem('todos')) || [];
} catch (e) {
  todos = [];
  localStorage.removeItem('todos');
}
```

Bukti. screenshots/bugs/bug-01-localstorage.png (sebelum perbaikan) dan screenshots/bugs/fix-01-try-catch.png (sesudah).



Gambar 10. Bug #1 sebelum perbaikan: SyntaxError akibat JSON.parse pada data korup



Gambar 11. Bug #1 sesudah perbaikan: pembacaan localStorage dibungkus try/catch

Alasan keputusan. Data localStorage bisa korup karena ekstensi browser, proses save yang terhenti (crash/tab ditutup paksa), atau skema data lama. Meskipun jarang terjadi pada pengguna normal, penanganan yang benar membuat aplikasi tidak pernah menampilkan layar kosong tanpa penjelasan.

Bug #2 — return di empty state memotong pembaruan UI

Gejala. Saat satu-satunya tugas yang selesai dihapus, daftar menjadi kosong (empty state muncul) tetapi tombol "Hapus Semua Selesai" **tetap aktif**. Klik tombol tidak melakukan apa pun — UX membingungkan.

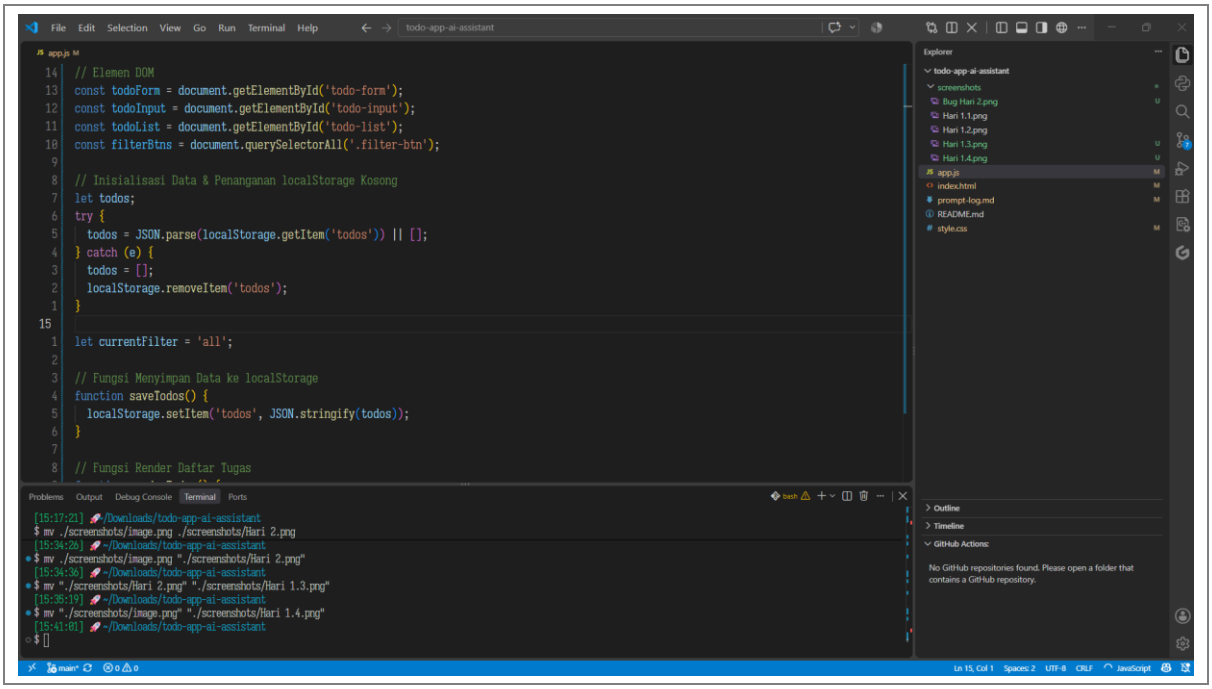
Penyebab. Pada integrasi empty state, kode awal menggunakan return di dalam if (filteredTodos.length === 0) { ... return; }. Akibatnya, dua baris di bawahnya — pembaruan counter dan status disabled tombol — **tidak pernah dieksekusi** saat daftar kosong.

Perbaikan. Mengubah return menjadi if / else sehingga seluruh bagian akhir renderTodos() selalu berjalan:

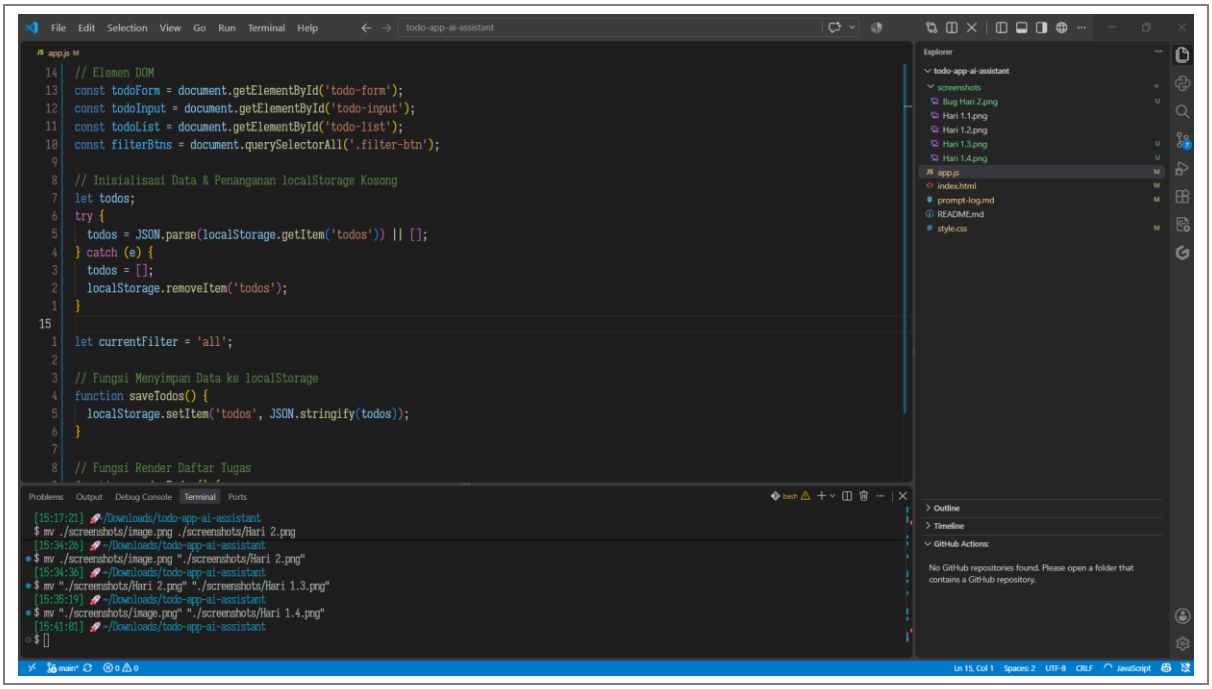
```
if (filteredTodos.length === 0) {
  // tampilkan empty state
} else {
  // render list
}

// selalu dieksekusi:
todoCounter.textContent = '...';
clearCompletedBtn.disabled = !todos.some(t => t.completed);
```

Bukti. screenshots/bugs/bug-02-return-empty-state.png (tombol tetap aktif padahal daftar kosong) dan screenshots/bugs/fix-02-return-empty-state.png (setelah perbaikan).



Gambar 12. Bug #2 sebelum perbaikan: tombol tetap aktif padahal daftar kosong



Gambar 13. Bug #2 sesudah perbaikan: tombol disabled saat tidak ada tugas selesai

Alasan keputusan. Penting untuk dicatat bahwa **bug ini bukan dari AI** — melainkan dari integrasi manual yang kurang cermat. Hal ini justru memperkuat argumen bahwa tanggung jawab akhir tetap di tangan pengembang, bukan pada AI.

Bug #3 — Counter tidak update saat tambah/toggle/hapus

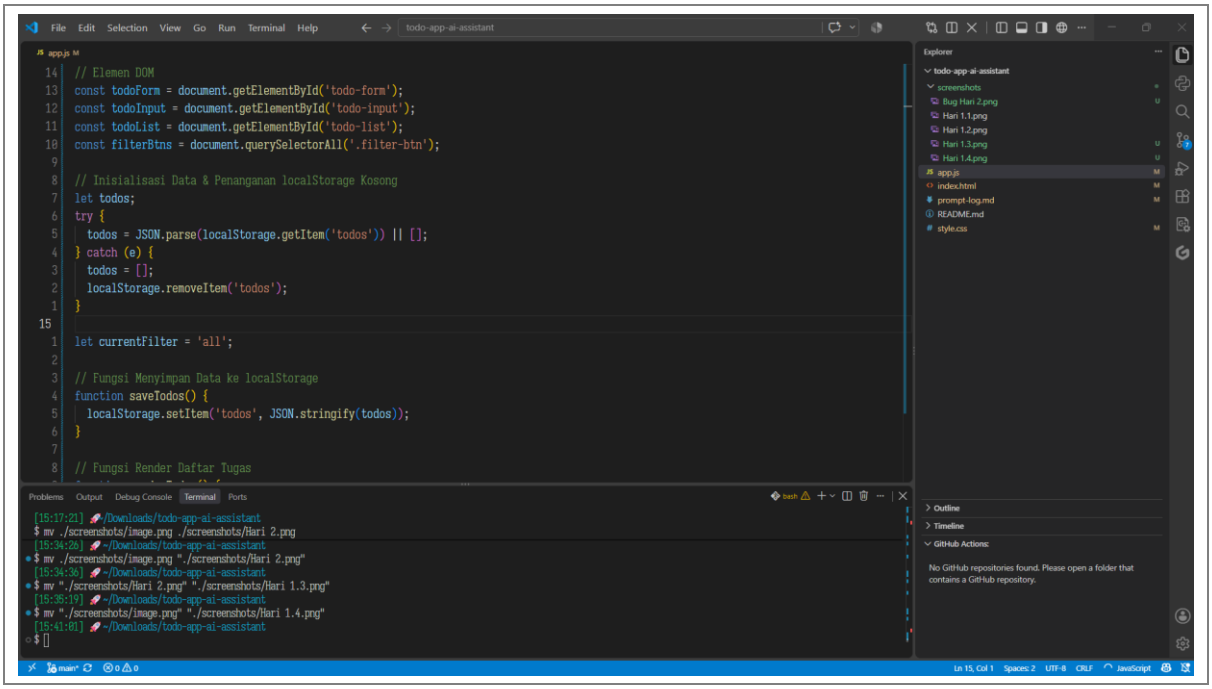
Gejala. Setelah menambahkan tugas, jumlah tugas muncul di list tetapi counter tetap menampilkan "0 tugas belum selesai dari 0 total". Counter baru berubah setelah halaman di-refresh.

Penyebab. Baris pembaruan counter awalnya ditempatkan **di luar** fungsi renderTodos(), sehingga hanya dieksekusi sekali saat halaman dimuat. Setiap kali renderTodos() dipanggil ulang setelah perubahan state, counter tidak ikut dihitung ulang.

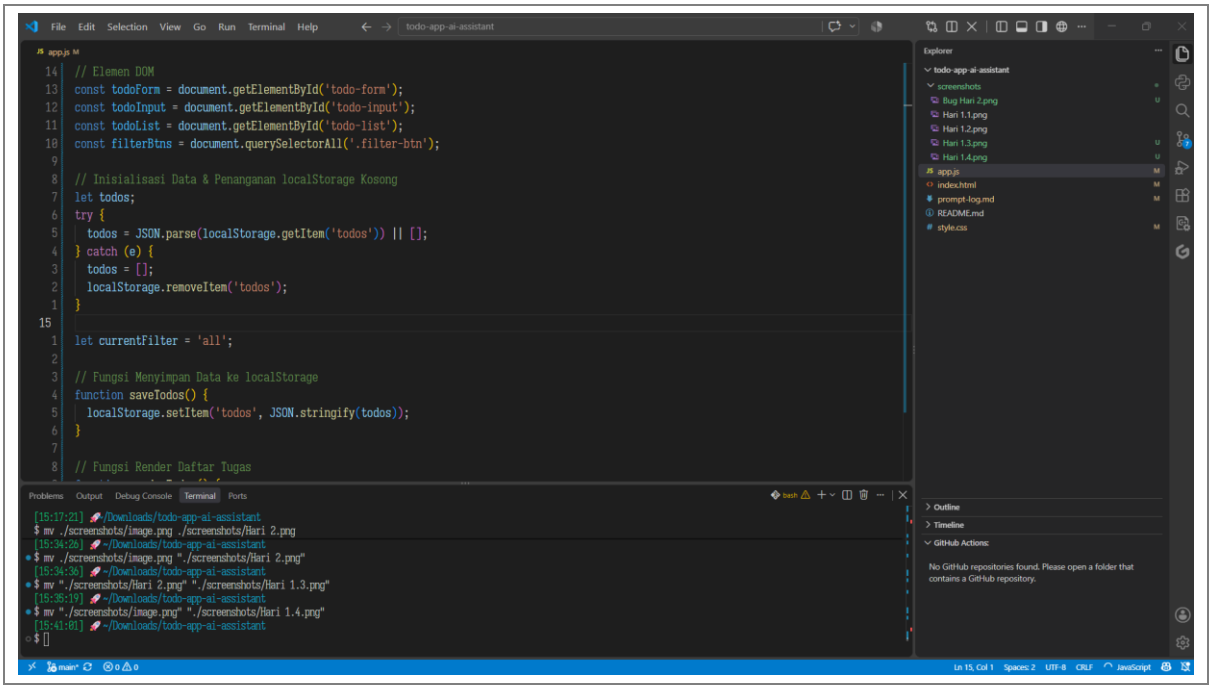
Perbaikan. Memindahkan pembaruan counter ke **dalam** renderTodos():

```
function renderTodos() {
  // ... render list ...
  const remaining = todos.filter(t => !t.completed).length;
  todoCounter.textContent = remaining + ' tugas belum selesai dari ' + todos.length + ' total';
  clearCompletedBtn.disabled = !todos.some(t => t.completed);
}
```

Bukti. screenshots/bugs/bug-03-counter.png (counter "0 dari 0" padahal list berisi) dan screenshots/bugs/fix-03-counter.png (setelah perbaikan, counter sesuai).



Gambar 14. Bug #3 sebelum perbaikan: counter "0 dari 0" padahal daftar berisi



Gambar 15. Bug #3 sesudah perbaikan: counter sesuai jumlah tugas

Alasan keputusan. Prinsip *single source of truth untuk render*: semua elemen UI yang bergantung pada state todos sebaiknya diperbarui di satu tempat. Ini mencegah bug di masa depan saat logika bertambah kompleks.

3.5 Keputusan yang Diambil di Luar AI

Beberapa keputusan tidak diambil dari output AI, melainkan dari evaluasi mandiri:

1. **Empty state** ("Belum ada tugas. Tambahkan tugas pertamamu!") — memberikan *feedback* visual saat daftar kosong.

2. **Counter tugas tersisa** — informasi status tanpa perlu menghitung manual.
3. **Konfirmasi hapus dengan jumlah** ("Hapus 3 tugas yang sudah selesai?") — AI awalnya hanya memberi pesan generik tanpa jumlah.
4. **Penempatan update counter di dalam renderTodos()** — untuk konsistensi dengan prinsip *single source of truth*.
5. **Penolakan saran AI** untuk menambahkan framework dan fitur prioritas/deadline pada iterasi 1 — di luar ruang lingkup tugas.

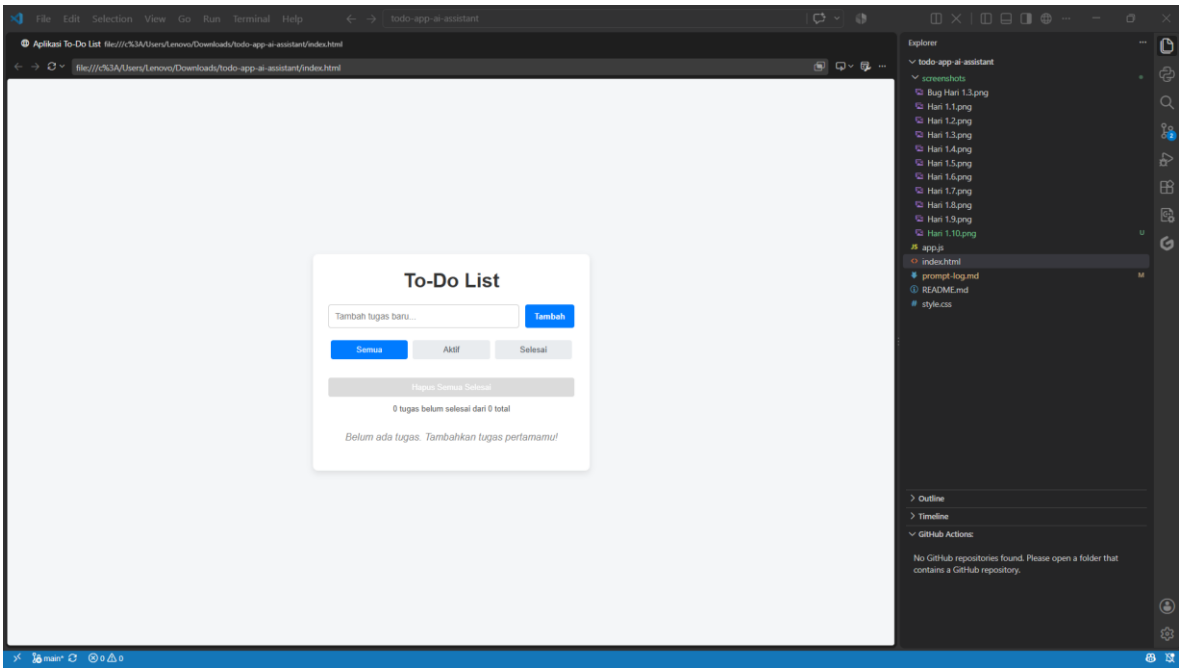
4. Pengujian

4.1 Metode

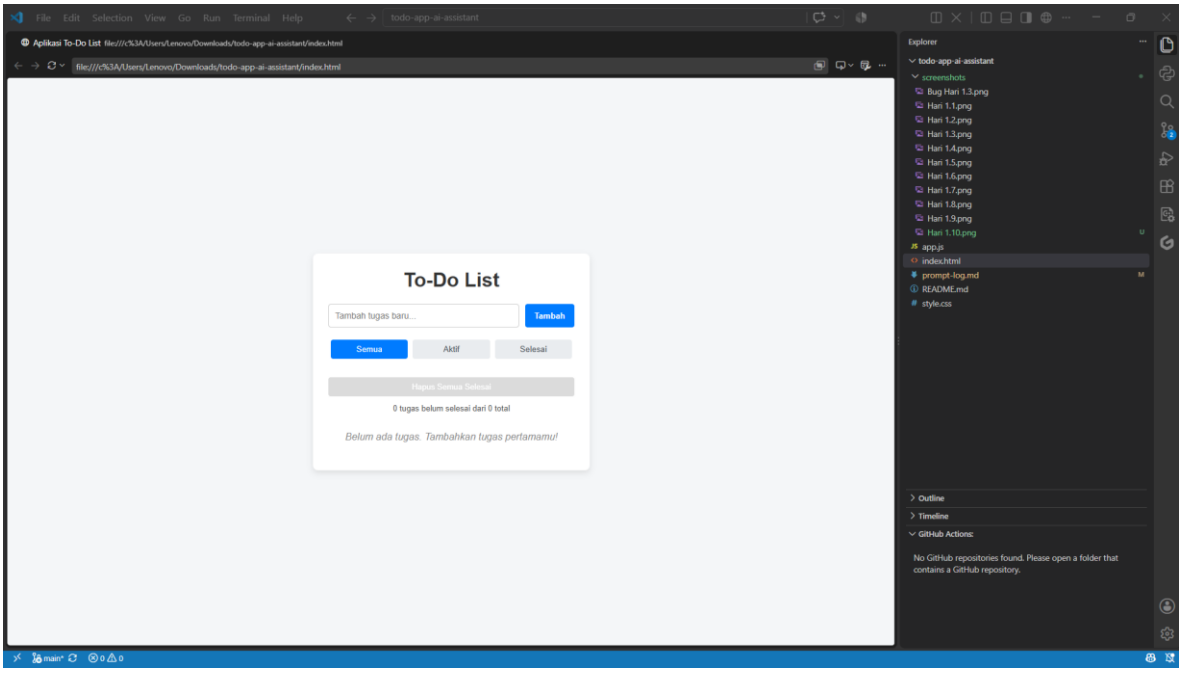
Pengujian dilakukan secara **manual end-to-end** di browser (Zen Browser dan VS Code Integrated Browser), mencakup 8 skenario fungsional dan 1 skenario kondisi tepi (*edge case*). Setiap skenario dijalankan dari state bersih (localStorage.clear() sebelum mulai) untuk memastikan hasil tidak dipengaruhi sisa data.

4.2 Hasil

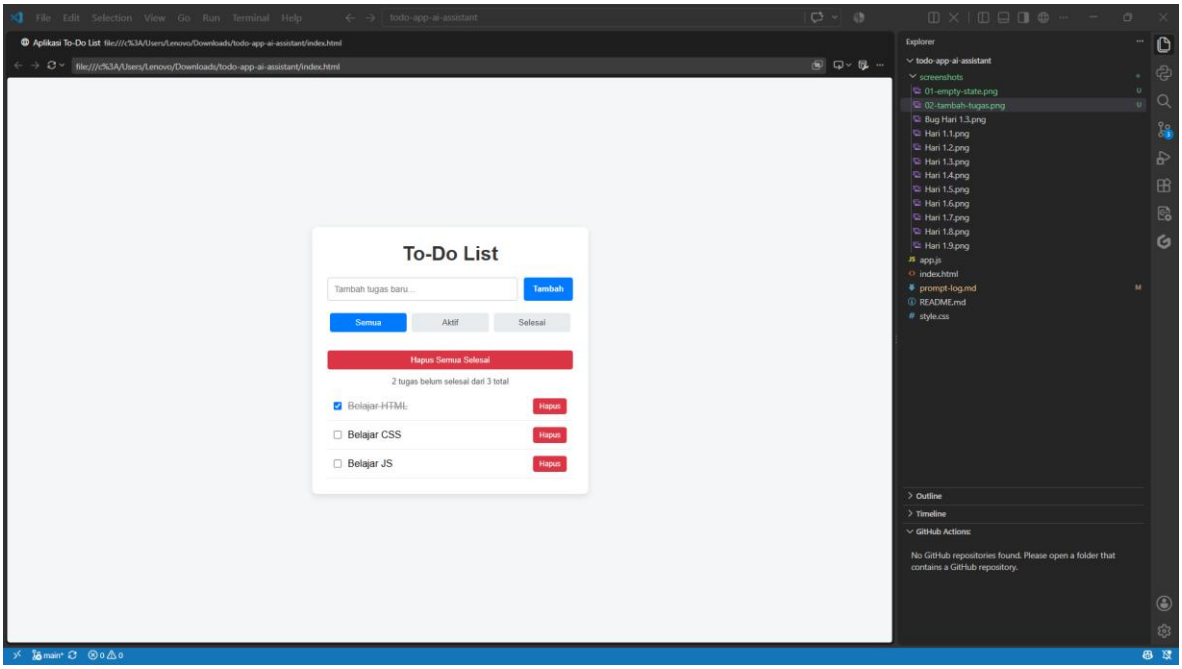
#	Skenario	Hasil	Bukti
1	Buka aplikasi dengan data kosong	Empty state tampil, counter "0 dari 0", tombol hapus-selesai disabled	screenshots/testing/01-empty-state.png
2	Tambah tugas "Belajar HTML"	Tugas muncul, counter "1 dari 1"	screenshots/testing/02-tambah-tugas.png
3	Tandai tugas selesai	Teks berubah coret, counter "0 dari 1", tombol hapus-selesai aktif	screenshots/testing/03-toggle-selesai.png
4	Klik filter Aktif	Hanya tugas belum selesai tampil	screenshots/testing/04-filter-aktif.png
5	Klik filter Selesai	Hanya tugas selesai tampil	screenshots/testing/05-filter-selesai.png
6	Refresh halaman (F5)	Data tetap sama, status selesai tetap	screenshots/testing/06-persist-refresh.png
7	Hapus semua tugas selesai	Hanya tugas selesai hilang, counter dan tombol diperbarui	screenshots/testing/07a-hapus-selesai-before.png, 07b-hapus-selesai-after.png
8	Simulasi data localStorage korup	Aplikasi tidak crash, list kosong, tombol disabled	screenshots/testing/08a-try-catch-before.png, 08b-try-catch-after.png



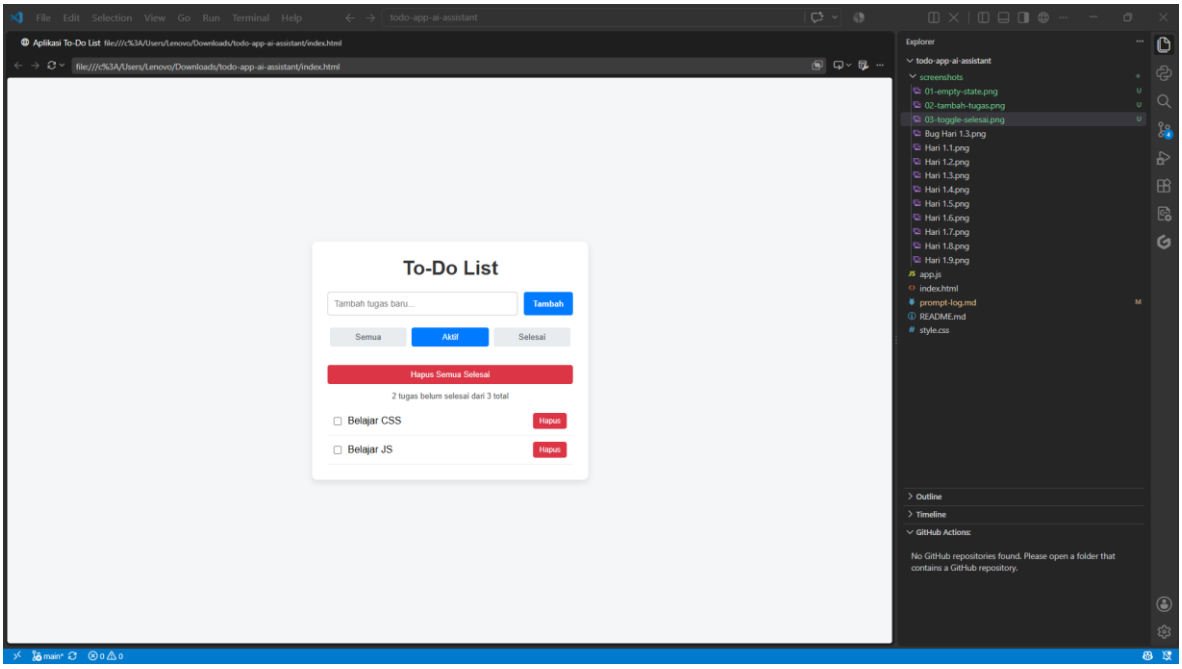
Gambar 16. Skenario 1: aplikasi dibuka dengan data kosong



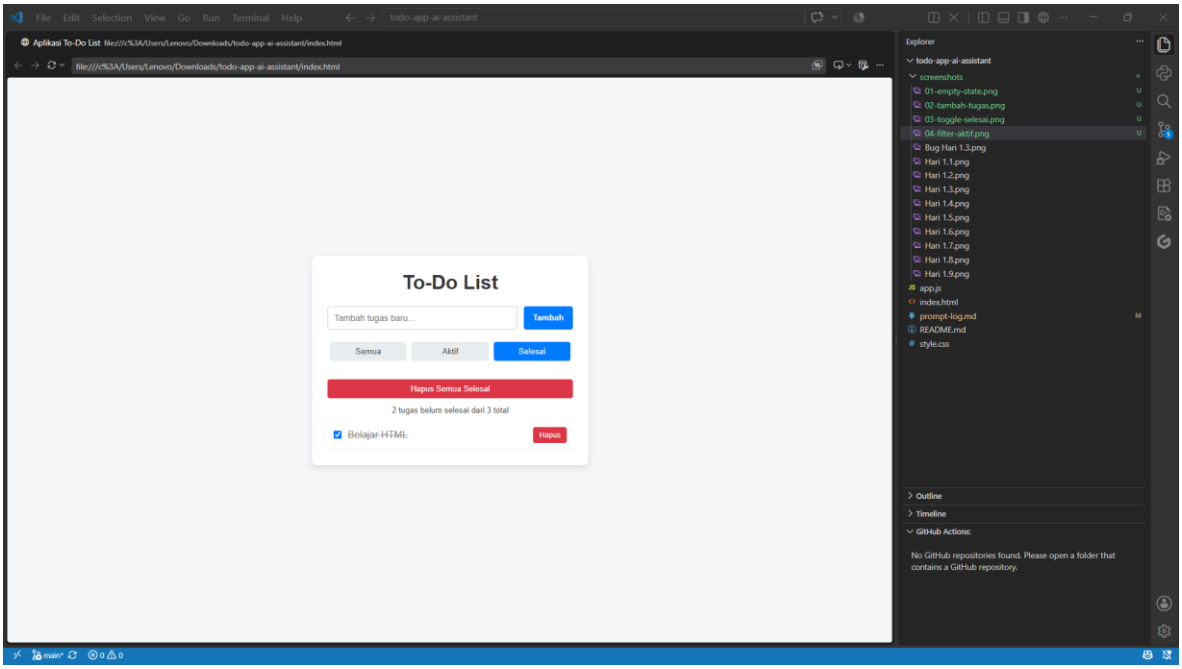
Gambar 17. Skenario 2: menambah tugas "Belajar HTML"



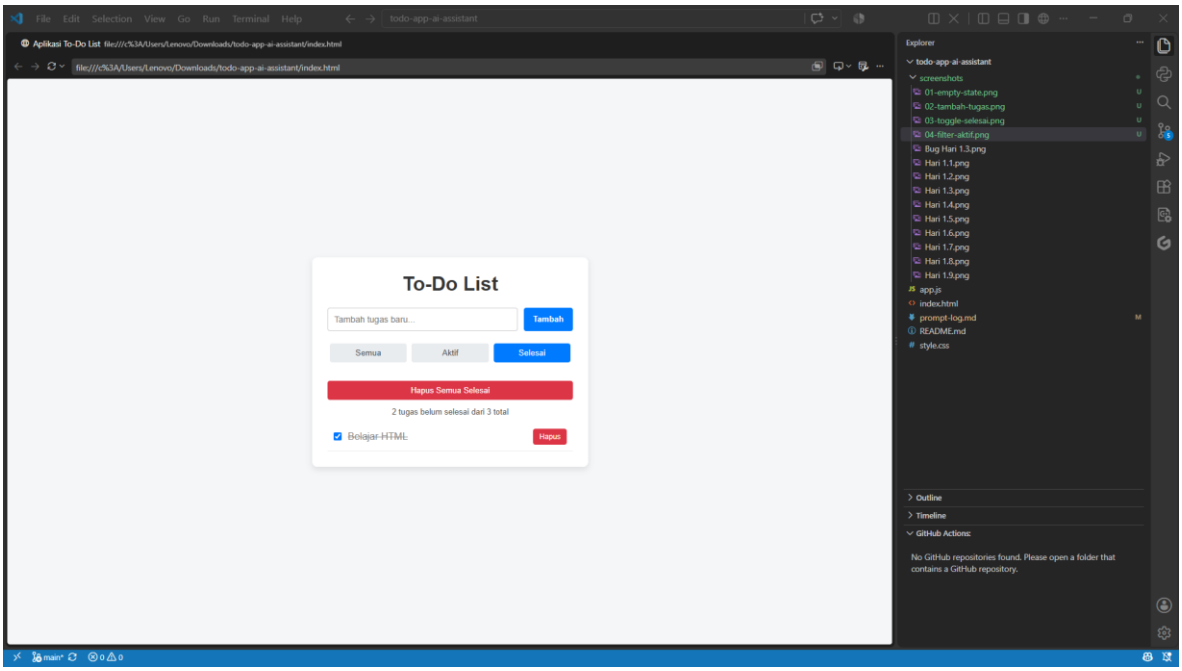
Gambar 18. Skenario 3: menandai tugas selesai



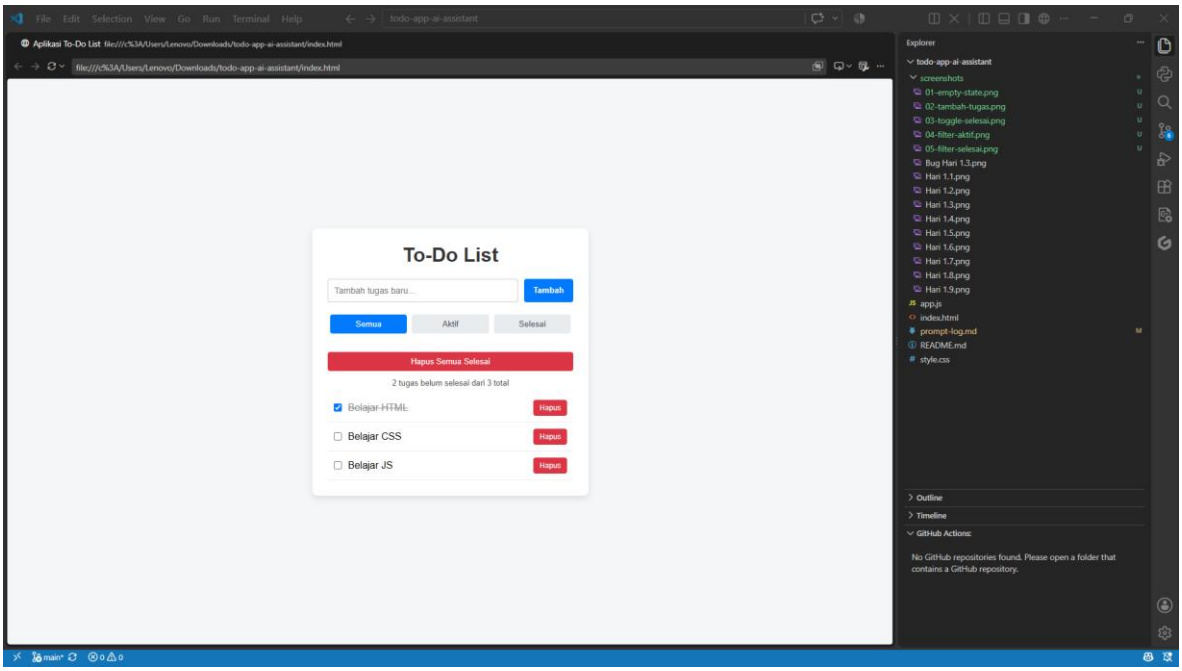
Gambar 19. Skenario 4: filter Aktif



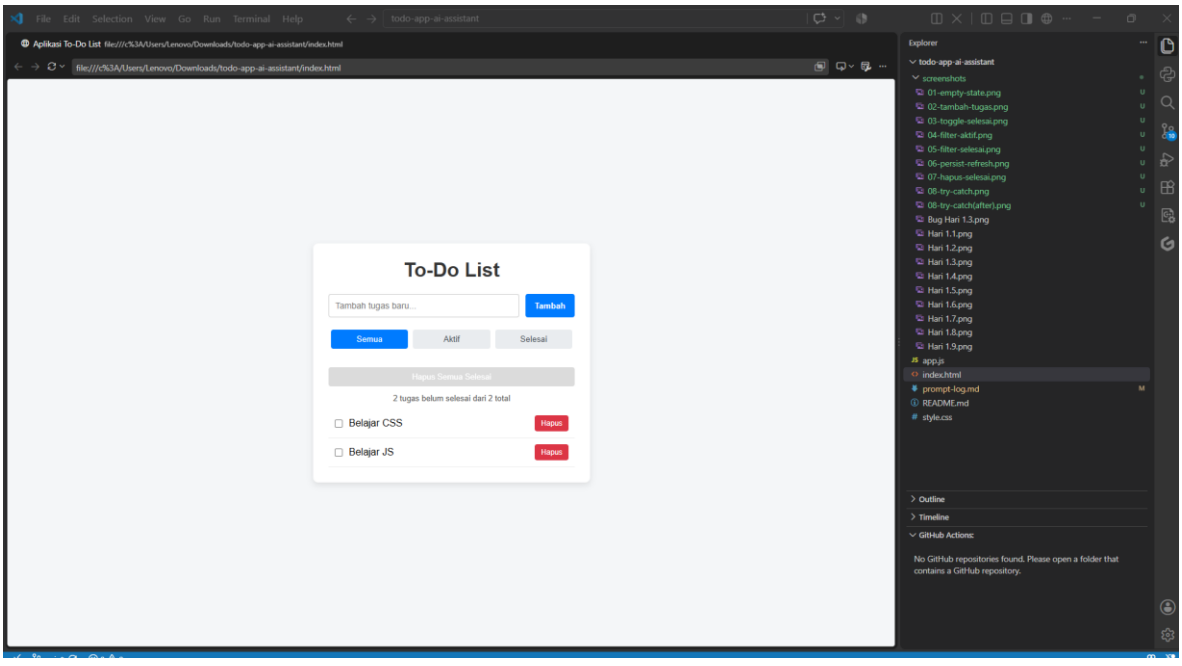
Gambar 20. Skenario 5: filter Selesai



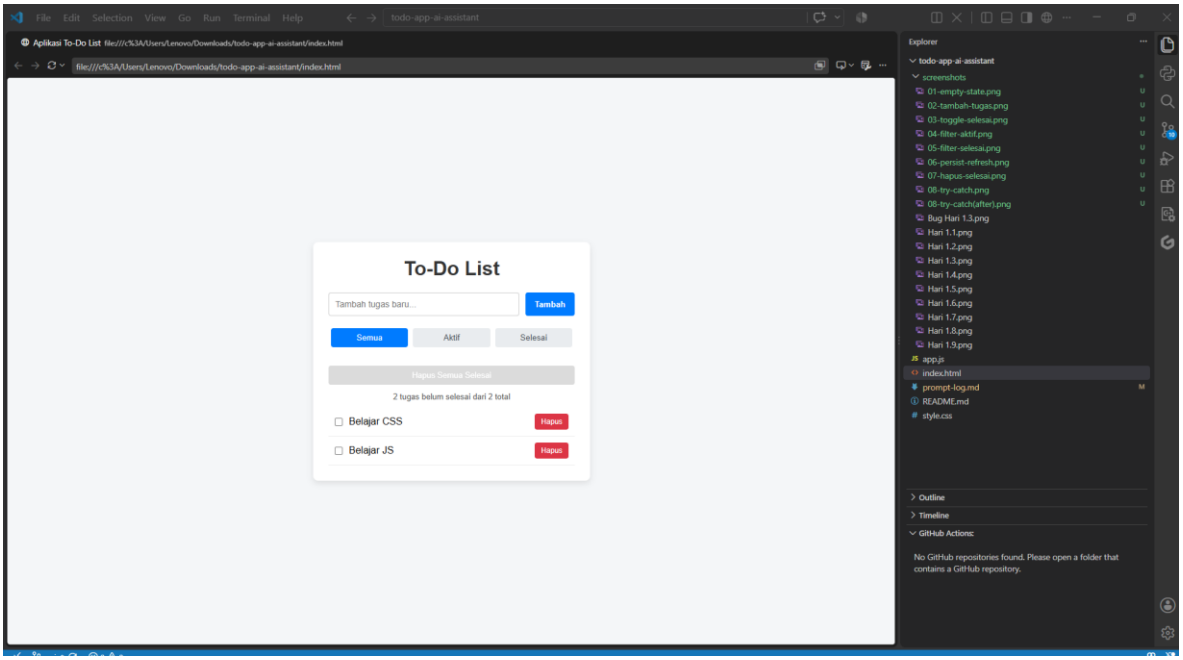
Gambar 21. Skenario 6: data tetap setelah refresh halaman (F5)



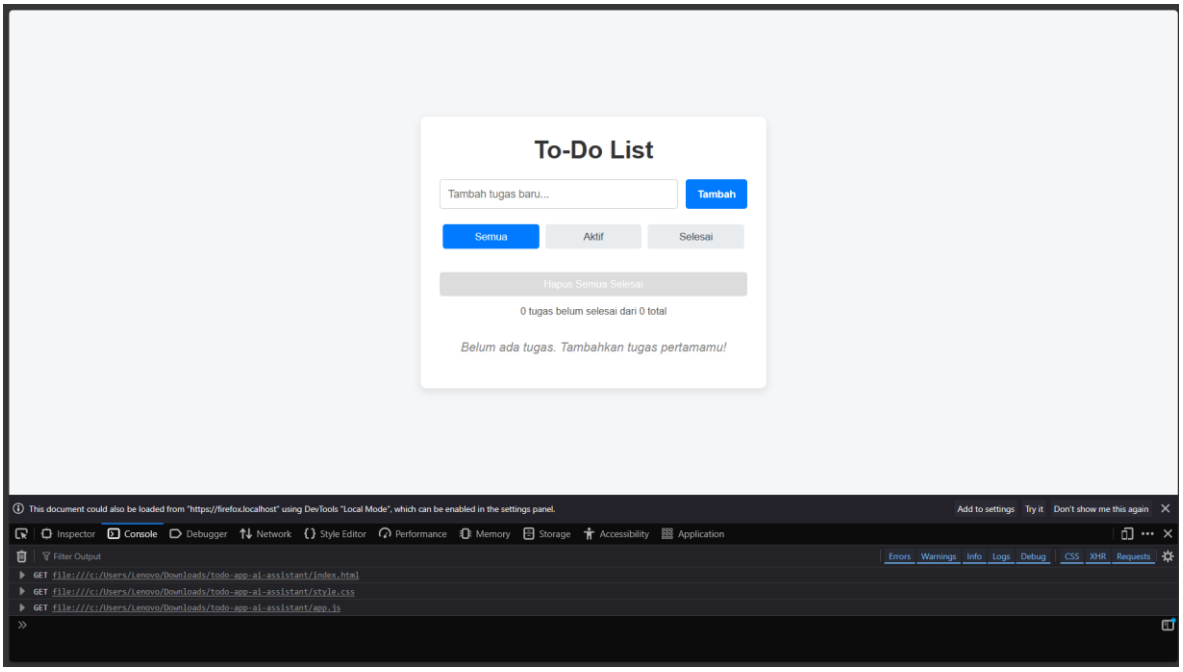
Gambar 22. Skenario 7: sebelum menghapus semua tugas selesai



Gambar 23. Skenario 7: sesudah menghapus semua tugas selesai



Gambar 24. Skenario 8: simulasi data localStorage korup (sebelum)



Gambar 25. Skenario 8: aplikasi tidak crash setelah penanganan data korup (sesudah)

Seluruh skenario **lolos** sesuai harapan. Dokumentasi visual lengkap tersimpan di folder screenshots/testing/.

4.3 Kesimpulan Pengujian

Aplikasi berjalan sesuai kebutuhan fungsional seluruh fitur wajib, tahan terhadap kondisi data localStorage yang korup, dan seluruh bagian UI (counter, tombol, empty state) tersinkronisasi

dengan state data. **Tidak ditemukan regresi** setelah penambahan fitur hapus-selesai dan perbaikan tiga bug pada Bab 3.4.

5. Kesimpulan

5.1 Ringkasan

Aplikasi web To-Do List sederhana berhasil dikembangkan menggunakan HTML, CSS, dan JavaScript vanilla, dengan bantuan Generative AI pada tahap rancangan dan implementasi awal. Selama proses, ditemukan tiga bug signifikan: dua bersumber dari integrasi manual (return yang memotong pembaruan UI, dan penempatan update counter yang salah), satu bersumber dari output AI yang kurang defensif (JSON.parse tanpa try/catch). Semua bug berhasil diidentifikasi, diperbaiki, dan didokumentasikan dengan bukti screenshot.

5.2 Refleksi Penggunaan AI

Pengalaman ini memberikan tiga pelajaran utama:

1. **AI mempercepat *boilerplate*, tetapi bukan pengganti verifikasi.** Iterasi 2 menghasilkan kode yang 90% langsung bisa dipakai, tetapi penanganan localStorage korup — yang krusial untuk ketahanan aplikasi — tetap terlewat.
2. **AI mengikuti instruksi teknis dengan baik, tetapi kurang peka terhadap UX.** Pada Iterasi 3, AI memenuhi semua syarat teknis (diff, disabled dinamis, konfirmasi), tetapi pesan konfirmasi yang dihasilkan masih generik tanpa menyebut jumlah tugas yang akan dihapus. Pengembang perlu menambahkan detail kecil seperti ini secara manual.
3. **Integrasi manusia–AI juga sumber bug.** Bug #2 dan #3 bukan berasal dari AI, melainkan dari penempatan kode yang kurang tepat saat mengintegrasikan fitur tambahan. Ini menegaskan bahwa tanggung jawab akhir tetap pada pengembang.

5.3 Saran Pengembangan

- **Migrasi ke framework** (React/Vue) jika fitur bertambah — state management akan lebih terstruktur dan meminimalisir bug sinkronisasi UI.
- **Sinkronisasi cloud** (Firebase, Supabase) agar data dapat diakses lintas perangkat.
- **Kategori dan prioritas tugas** untuk aplikasi yang lebih kompleks.
- **Automated testing** (Jest, Vitest) untuk mencegah regresi saat kode berkembang.
- **Penambahan konfirmasi hapus per-tugas** — saat ini konfirmasi hanya ada pada hapus massal.

Lampiran

- **prompt-log.md** — log lengkap prompt, output AI, evaluasi, bug, dan perbaikan.
- **screenshots/prompts/** — dokumentasi visual prompt dan output AI (Iterasi 1–3, total 9 gambar termasuk lanjutan output iterasi 2 dan 3).
- **screenshots/bugs/** — bukti kondisi bug sebelum perbaikan dan setelah perbaikan (3 bug, 6 gambar).
- **screenshots/testing/** — hasil uji end-to-end 8 skenario (10 gambar, termasuk sebelum/sesudah untuk skenario 7 dan 8).
- **README.md** — deskripsi singkat repository.